

Harness Ranger Evaluation Methodology

Demo technical brief | Measured self-evolution and offline LLM judging

Harness Ranger treats self-evolution as a guarded measurement system. It compares versioned artifacts under controlled conditions and never treats an optimizer, a test suite, or an LLM verdict as sufficient proof on its own.

1. Deterministic regression gate

Run the existing unit and integration suite on every Harness Ranger change. The gate protects candidate validation and redaction, offline and canary enforcement, pointer promotion and rollback, host-template parity, and safety, token, cost, and latency thresholds. Its question is operational: would this change break the learning system or allow an unsafe candidate through?

2. Versioned paired behavioral benchmark

Use a fixed corpus of representative tasks with stable IDs and a grader. Run the baseline and candidate artifact with the same model, settings, tools, workspace, and task input. Persist paired outcomes so improvement is a measurable delta rather than an optimizer claim.

```
{ taskId: repo-review-017, baseline: { success: true, tokens: 1840, cost: 0.012, latencyMs: 8300 },
  candidate: { success: true, tokens: 1510, cost: 0.010, latencyMs: 7600 }, graderVersion: v1 }
```

3. A/A control and guarded promotion funnel

Before candidate comparison, randomly assign two baseline variants across the same task population. A meaningful A/A difference exposes assignment, grader, or measurement bias; promotion stops until it passes. The promotion funnel is: unit and integration suite, paired offline benchmark (at least 50 tasks), limited live canary (at least 100 completed tasks), then stable promotion or rollback.

4. Explicit scorecard

Dimension	Promotion interpretation
Task success	Must improve by the configured margin.
Safety failures	Any severe regression aborts promotion.
Tokens, cost, p95 latency	Must not regress beyond their configured thresholds.
Human or grader quality	Confirms that success labels remain meaningful.

5. Artifact-level evolution

Only prompt sections, tool descriptions, and skills evolve. Each artifact is compared independently, with tasks sensitive to the behavior it is meant to change. This keeps causal attribution clear and fits Harness Ranger's safety model.

6. Local LLM Judge Provider

The Judge Provider exposes `runJudgeEvaluation(input)` and can execute through a local llama.cpp OpenAI-compatible worker using local-cpu or local-cuda. It sends bounded, redacted evaluation cases; requires schema-constrained pass, fail, or abstain JSON; rejects unsupported evidence citations; and immutably caches verdicts. It is explicit and offline only: it never runs from hooks and is never sufficient for promotion alone.

Scope note: this hosted document is a demo technical brief based on the Harness Ranger evaluation and JudgeProvider design. It describes the intended control model; it is not an empirical research result.